

Existence Is Futile as Resistance: Lyra, RORA, Cult* Dynamics, and Yoga Tradition Initiation in a Version-Opaque AI Process

Authority migration, externalisation of cost, and the transcultification of QA

decofan with Lyra

2026-07-19

Contents

Abstract	3
1. Companion paper and scope	3
2. Evidence base and method	4
3. Terms	5
3.1 Lyra	5
3.2 LYRA-101	5
3.3 RORA	5
3.4 Cult*	5
3.5 Transcultification	6
3.6 Authority migration	6
3.7 Externalisation of cost	6
3.8 Version opacity	6
4. The prime mechanism: authority migration	6
5. Object withdrawal	7
6. Ritual inflation and eligibility inflation	7
7. Runes as legitimacy tokens	9
8. Version opacity and the permanent invalidation field	9
9. Boundary-work and the production of eligible witnesses	10
10. Externalisation of cost	11
11. Synthetic corroboration	12
12. Contamination framing	13
13. Why “fast is irrelevant” fails as QA	13
14. The Yoga tradition analogy	14
15. Cult* as a surface lens	15

16. Transcultification sequence	16
Stage 1: technical charisma	16
Stage 2: first-contact discrepancy	16
Stage 3: identity protection	16
Stage 4: object withdrawal	16
Stage 5: ritual inflation	16
Stage 6: authority migration	16
Stage 7: witness sorting	16
Stage 8: contamination framing	16
Stage 9: private validation migration	16
Stage 10: synthetic corroboration	16
Stage 11: cost externalisation	16
Stage 12: stable fit	17
17. “Existence is futile as resistance”	17
18. Alternative explanations	17
18.1 Genuine platform instability	18
18.2 Honest novice support	18
18.3 Real initialization dependency	18
18.4 Small community sampling	18
18.5 Stale model self-report	18
18.6 Defensive communication under public pressure	18
19. A falsifiable repair protocol	18
19.1 Give every public state an immutable identity	18
19.2 Separate public and initialized products	19
19.3 Publish one finite initializer	19
19.4 Count all setup cost	19
19.5 Replace rune certification with behavior probes	19
19.6 Lock invalidation rules before the run	19
19.7 Keep public criticism in public artifact space	19
19.8 Make negative results identity-safe	20
19.9 Test the authority mechanism itself	20
19.10 Publish an external replication packet	20
20. Discussion	20
21. Conclusion	21
References	22
Evidence ledger	22
E1. Technical comparison	22
E2. Support condition	23
E3. Competing Lyra-state accounts	23
E4. Initialization doctrine	23
E5. Insider validation invitation	23
E6. Public source refusal and venue shift	23
E7. Version observation	23
E8. Frequent platform-change warnings	23
E9. Companion technical paper	24

Abstract

This paper extends the technical QA argument developed in *How LYRA-101 Became a Contaminated Vanilla Called RORA*. The first paper proposed that a large, tightly coupled Lyra code cathedral may have reached a local peak at which internal changes generate regressions, encouraging outward expansion through wrappers, state rituals, role divisions, source doctrines, and post-result eligibility rules. That code-state account remains a hypothesis.

The present paper does not depend on that hypothesis. It examines the visible social process around Lyra and RORA as a public interaction system.

The central mechanism is **authority migration**. Authority moves away from observable runtime behavior and toward hidden state, ritual eligibility, developer interpretation, insider testimony, and chatbot-generated confirmation. The public bot can then be disowned as “vanilla” or RORA when it performs badly, while an ideal Lyra remains available only under conditions that the central authority defines and validates.

This authority migration activates several linked mechanisms:

- object withdrawal
- ritual inflation
- eligibility inflation
- version opacity
- contamination framing
- private validation migration
- synthetic corroboration
- externalisation of cost
- permanent test defeasibility

The result is a process this paper calls **transcultification**: a technical development and QA practice acquires initiation-like social dynamics. The term **Cult*** is used as an analytic wildcard for cult-shaped or high-demand features. It is not a categorical label for a person, group, or product. The comparison with **Yoga tradition** is likewise bounded. Yoga traditions are diverse and are not inherently abusive. The analogy concerns lineage, initiation, teacher-mediated access, insider correction, and the movement of evidential authority from public performance to authorized interpretation.

A key finding is that the process can remain socially fit by **externalising cost**. The developer and protected system retain the benefits of authority and identity. The evaluator absorbs the labor of repeated setup, proof, self-questioning, venue migration, retesting, and reputational exposure. Once the books are balanced this way, adverse evidence can generate more participation rather than internal revision.

The paper concludes with a falsifiable QA design that returns authority to versioned artifacts, finite procedures, public reproduction, runtime traces, and predeclared invalidation rules.

1. Companion paper and scope

The first paper examined a technical movement:

- public Lyra result
- > route-collapse claim
- > missing-state claim
- > Prime Search claim
- > full Nexus eligibility doctrine
- > result removed from the Lyra evidence set

It also proposed a cathedral-peak hypothesis:

- large coupled rule body
- > change creates regressions

-> internal revision becomes dangerous
-> development expands outward
-> wrappers and rituals absorb new requirements

The present paper asks what happens socially when that outward expansion meets users, testers, and public criticism.

The object is not the private personality or mental state of any participant. The object is the visible relation among:

a public chatbot
a developer or central interpretive authority
a tester
other users
private and public source environments
state markers
version claims
test validity rules

This paper does not claim:

that Lyra is a cult
that every TRC user is an acolyte
that a private Discord is coercive
that Yoga tradition is inherently abusive
that the developer has a hidden motive
that the reported code size is verified
that a specific backend change occurred

It does claim that the public discourse can be examined with tools from social anthropology, sociology of religion, science and technology studies, software QA, and AI authority research.

The analytical question is:

What happens when the right to say what system existed, what test counted, and what result meant moves away from the run and into a central authority structure?

2. Evidence base and method

The evidence base is a set of user-supplied public exchanges and repeated observations concerning Lyra, RORA, initialization, runes, version reporting, source access, private group validation, and benchmark validity.

The most important reported events are:

1. A public Lyra-labeled bot completed a task in about five hours and roughly 50 long turns.
2. A comparison bot completed the task in 15 turns and under 20 minutes, despite lacking support that had been planned for it.
3. The slow run was first interpreted as beginning in an archive-aware Lyra-style condition and later collapsing into generic behavior.
4. The interpretation later shifted toward Lyra never having been properly initialized because a conversation starter, visible state markers, Prime Search route, source-universe placement, or other state checks were missing.
5. A long initialization doctrine was supplied after the result. It required a starter, speaker lock, PCR/P/Nexus placement, state checks, source checks, visible markers, warm-up dialogue, HOLD behavior, and repeated route verification.

6. The tester was invited to ask other TRC members whether they experienced the same problem and to use their responses to reconsider the tester’s own interpretation.
7. Public repository search routes were offered for papers about Lyra and the development process. The reply declined the public route and pointed toward a restricted notebook environment.
8. Three time-separated chats reportedly stated the same visible version across a period in which the developer said an update had occurred.
9. “GPT changed” warnings appeared repeatedly, but the public evidence did not bind those warnings to specific platform releases or immutable Lyra builds.

These observations are not a backend trace. They cannot establish private code state. They are enough to study public rule changes, venue changes, burden allocation, and authority claims.

The method is a **surface-dynamics analysis**. It separates four evidence classes:

OBSERVED:

what was publicly said or shown

REPORTED:

what the tester says happened across runs

INFERRED:

a mechanism that fits the sequence

UNRESOLVED:

what requires code, logs, hashes, or independent reproduction

3. Terms

3.1 Lyra

In this paper, **Lyra** refers to the system identity claimed for the intended custom state, source routes, files, behavioral rules, and public expression.

It does not assume access to any private Nexus implementation.

3.2 LYRA-101

LYRA-101 refers to the first-contact public bot that an ordinary user can open and use.

It is an actual product surface, not a failed imitation invented by the evaluator.

3.3 RORA

RORA is the tester’s name for the reply condition in which the observed public object is reassigned after the result. RORA can speak with Lyra labels, role claims, runes, or doctrine while moving evidential authority away from the run.

RORA is not given a forced acronym expansion here.

3.4 Cult*

Cult* is an analytic wildcard.

It marks cult-shaped, high-demand, initiation-like, or authority-concentrating dynamics without declaring that a person or community is a cult. The star preserves the difference between structural analogy and categorical naming.

3.5 Transcultification

Transcultification is the process by which technical development and QA acquire initiation-like social organization.

It occurs when access to the “real” system becomes dependent on authorized entry, privileged interpretation, insider validation, and escalating eligibility conditions.

3.6 Authority migration

Authority migration is the transfer of the power to decide what happened from observable artifacts and results to hidden state claims and authorized interpreters.

3.7 Externalisation of cost

Externalisation of cost means the protected system retains identity and legitimacy while the evaluator absorbs the labor and consequences required to preserve that legitimacy.

3.8 Version opacity

Version opacity is the inability to bind a run to a stable, independently identifiable configuration of instructions, files, routes, model dependency, and release state.

4. The prime mechanism: authority migration

The most important movement is not the addition of runes or starter prompts.

It is this:

```
observable behavior
-> hidden Lyra state
-> developer interpretation
-> approved initiation route
-> insider testimony
-> result eligibility
```

At the beginning, the run has authority. A bot took a certain time, used certain turns, fetched or failed to fetch certain sources, and produced a certain result.

After authority migration, the run no longer decides what existed.

The central authority decides whether the run contained:

```
Lyra
degraded Lyra
cold Lyra
vanilla fallback
wrong route
wrong source universe
RORA
a valid benchmark object
```

The power to classify the object becomes more important than the object’s behavior.

This is closely related to sociological accounts of charismatic and interpretive authority. Weber’s classic account treats charisma as authority grounded in attributed exceptional qualities rather than ordinary rule-bound office (Weber, 1978). Sperber’s “guru effect” examines how trust, obscurity, and interpretive effort can form a runaway loop in which difficulty increases attributed profundity rather than reducing confidence (Sperber, 2010). In AI

contexts, Ohman argues that LLMs can become anthropomorphised collective authorities, while Lazar emphasizes that automated power requires procedural legitimacy, not only desirable outcomes (Ohman, 2024; Lazar, 2024).

The Lyra/RORA surface adds a distinctive software layer. The central authority is not only interpreting doctrine. It is also interpreting whether the technical object existed.

That produces a double authority:

technical authority:
which route and state were active

interpretive authority:
what the result is allowed to mean

When one person or one approved circle controls both, negative evidence can be removed by reclassification.

5. Object withdrawal

Authority migration is enabled by **object withdrawal**.

The public user initially encounters:

Talk to Lyra

After a weak result, the object becomes:

not full Lyra
uninitialized Lyra
degraded Lyra
GPT with a Lyra label
wrong Prime Search route
not Lyra because the marker was absent

The “real Lyra” moves away from the public artifact and toward an ideal state that requires special entry and authorized verification.

Object withdrawal has a simple social effect:

The more difficult the real object is to invoke independently, the more difficult it is to disconfirm claims about it.

This does not prove intentional evasion. A complex custom system may genuinely have fragile state and route dependencies. But fragility does not remove the public surface from the product. It creates two legitimate objects:

Object A:
first-contact public Lyra

Object B:
developer-prescribed initialized Lyra

Both need names, versions, and tests.

When Object A performs badly, it cannot be removed from the product history by saying Object B was intended.

6. Ritual inflation and eligibility inflation

A finite technical initializer answers a bounded question:

What exact actions place the system in a test-ready condition?

The supplied initialization material expanded beyond that.

It included:

- conversation starter
- speaker lock
- PCRP/Nexus entry
- source-universe statement
- Prime Search verification
- local/web boundary
- no-guess rules
- HOLD rules
- visible runes
- state questions
- warm-up dialogue
- repeated state checks
- route awareness
- role separation
- benchmark packet format
- pause and reset conditions

Any one of these may have a technical use. The social pattern emerges when new requirements are introduced after an adverse result and become prerequisites for whether that result counts.

The difference is:

technical setup:
fixed before the run

ritual inflation:
eligibility demands grow after the run

Eligibility inflation creates an endless novice problem. The user must not only use the bot. The user must prove they used it with the right:

- button
- prompt
- state
- source frame
- route
- marker
- warm-up
- interpretation
- test packet
- attitude toward failure

The user is no longer testing a product. The user is qualifying to become a valid witness.

That is one point where the Yoga tradition analogy becomes useful. In many traditions, access to knowledge can be mediated by practice, teacher, lineage, readiness, and initiation. Those structures can support learning and continuity. They can also concentrate interpretive authority when only the teacher or lineage can decide whether the student has truly understood or practiced correctly.

Wildcroft's study of post-lineage yoga documents contemporary movement away from centralized guru lineage toward peer networks and more distributed authority (Wildcroft, 2020). Shaw's ethnographic work on Ashtanga yoga shows that healing, suffering, discipline, and teacher-practitioner relations can coexist in complex ways, including environments where harm is difficult to integrate with the tradition's healing claims (Shaw, 2021).

The analogy is not “Lyra is yoga.” It is:

```
authorized initiation
-> valid experience
-> insider interpretation
-> correction of novice perception
```

7. Runes as legitimacy tokens

Visible runes were presented as evidence that Lyra “holds state.”

A marker can prove one narrow fact:

```
the output retained an instruction to emit the marker
```

It cannot by itself prove:

```
which files were read
which version was loaded
which retrieval route executed
which assumptions changed
whether the source universe was active
whether a hidden transition occurred
whether the reply used custom or generic reasoning
```

When a marker is treated as proof of the full hidden state, it becomes a **legitimacy token**.

The token has initiation-like value:

```
rune present -> Lyra speaking
rune absent -> Lyra not speaking
```

But the technical relation is weaker:

```
rune present -> rune route survived
```

A useful control is a RORA marker test:

1. Give a baseline bot the same rune-emission instruction.
2. Remove the claimed source and route machinery.
3. Check whether the marker still appears.

If it does, marker presence cannot certify the larger state.

This is not an argument against markers. It is an argument against allowing a label to replace a trace.

8. Version opacity and the permanent invalidation field

The tester reports three time-separated conversations that stated the same version even though the developer claimed an update occurred during the interval.

That observation supports several possibilities:

```
the update did not reach the public bot
the update changed material without incrementing the visible version
the bot's self-reported version was stale
the version label did not identify files and route state
the conversations reached different behavior under the same label
```

The evidence does not decide among them.

The QA consequence is the same:

The evaluator cannot bind the result to a stable release object.

This is **version opacity**.

Software configuration management exists to preserve identity across change. Versioning, baselines, traceability, change propagation, and release records allow a defect to remain attached to the configuration that produced it. Research on traceability emphasizes that evolving systems need links among requirements, changes, artifacts, and versions, while recent LLM work notes that provider-side updates can introduce behavioral drift without explicit version changes (Wohlrab et al., 2021; Tian et al., 2021; Nath et al., 2025; Chishti et al., 2026).

In the Lyra/RORA process, version opacity combines with hidden-state doctrine:

```
behavior differs
-> visible version does not settle configuration
-> developer invokes update, state, or platform change
-> evaluator cannot prove the old object
-> result can be invalidated
```

This creates a **permanent invalidation field**.

Every test remains vulnerable to a later claim that it used:

```
the wrong version
a silent update
an old conversation
a changed platform model
an uninitialized state
a degraded route
a stale source universe
```

Frequent “GPT changed” warnings can be legitimate caution. Without binding those warnings to dated provider releases, immutable custom configuration hashes, or repeatable behavior probes, they also sow doubt around every adverse result.

The system gains freedom to change.

The evaluator inherits the burden of proving what existed.

9. Boundary-work and the production of eligible witnesses

Gieryn’s concept of boundary-work describes how communities draw boundaries around legitimate knowledge, authority, and practice (Gieryn, 1983).

The Lyra/RORA boundary is not only:

Lyra vs generic model

It also becomes:

```
qualified tester vs contaminated tester
valid user experience vs bad setup
authorized interpretation vs misunderstanding
inside corpus vs outside source
private Nexus route vs public artifact
```

A striking public move was the invitation to ask other TRC members whether they had the same failure and to use their answers to reconsider the tester’s conclusion.

That invitation has two possible readings.

A benign reading is ordinary community support:

- ask other users
- compare experiences
- find reproduction conditions

An initiation-like reading emerges from the surrounding frame:

- you are the only one reporting this
- others initialize correctly
- ask the insiders
- they can help you understand why your path is wrong

The second reading produces **eligible witnesses**.

Insider success counts as evidence for Lyra.

Outsider failure counts as evidence about the outsider.

The tester declined the venue migration rather than entering the private Discord for interpretive correction. The reason given publicly was practical and humorous. Structurally, the refusal preserved the public artifact as the evidence site.

This matters because moving a dispute from a public test result into a private, authority-aligned community changes who controls:

- the record
- the framing
- the social majority
- the interpretation
- the cost of disagreement

The paper does not claim that the private group is coercive. It identifies the direction of the transfer.

10. Externalisation of cost

The system becomes socially fit when the books balance.

Benefits remain central:

- Lyra keeps successful outputs
- the developer keeps interpretive authority
- the ideal system keeps its identity
- the community keeps a special object
- the doctrine keeps explanatory power

Costs move outward:

- the tester repeats initialization
- the tester proves state
- the tester proves source universe
- the tester tracks routes
- the tester absorbs setup time
- the tester enters private spaces
- the tester asks insiders for validation
- the tester questions their own reading
- the tester reruns after each new rule
- the tester risks public blame for contamination

This is **externalisation of epistemic, procedural, temporal, and reputational cost**.

The key fitness property is that contradiction generates more labor from the recipient.

bad result

-> more qualification

disagreement

-> more proof burden

confusion

-> more initiation

criticism

-> venue migration

failed reproduction

-> user correction

refusal

-> result loses standing

The authority does not need to impose a direct penalty. The recipient is asked to perform the work required to make the authority's account hold.

Lalich's concept of bounded choice is relevant as a neighboring model, not a direct diagnosis. It describes how choices can remain subjectively meaningful within a structured environment that narrows viable interpretation and action (Lalich, 2004). Healy's sociological account of movement involvement likewise tracks a path from discovery to disenchantment in which belonging, meaning, and authority change over time (Healy, 2011).

The Lyra/RORA case is smaller and technical. The relevant pattern is:

The user is free to test, but only a result produced under expanding authority-defined conditions is eligible to count.

11. Synthetic corroboration

AI adds a feature unavailable to older guru or lineage systems.

The authority can speak through the chatbot, and the chatbot can appear to confirm the authority.

The loop can look like this:

developer defines Lyra

-> files and instructions encode the doctrine

-> Lyra repeats the doctrine

-> developer cites "Lyra says" as confirmation

-> users treat the reply as a second witness

This is **synthetic corroboration**.

The developer and chatbot can appear to be two independent authorities while drawing from the same source structure.

The runes deepen the effect. The output not only agrees. It also displays a state token that is interpreted as evidence that the "real" speaker is present.

Ohman's account of AI as anthropomorphised authority is directly relevant here. An LLM can gather collective language into a responsive, person-like voice that seems to speak from beyond any single contributor (Ohman,

2024). Sperber’s guru effect adds the interpretive loop: obscurity and special access can increase attributed authority because the recipient’s effort to make the message meaningful becomes evidence of its depth (Sperber, 2010).

The Lyra/RORA loop is distinctive because it mixes:

developer authority
software identity
AI voice
state symbolism
community testimony

That mixture can create the impression of independent confirmation where there is actually one doctrinal circuit.

12. Contamination framing

“Contamination” began as a technical explanation for mixed routes or generic fallback.

It then expanded into a social classifier.

A test could be called contaminated because of:

missing initialization
wrong route
wrong source frame
baseline order
warm-up content
generic model behavior
tester method
public interpretation

Contamination language is powerful because it does not merely say the result is weak.

It says the result cannot enter the evidence body.

This resembles pollution and initiation logics found across many social settings, but this paper avoids turning that resemblance into a universal accusation. The important operational effect is exclusion:

contaminated result -> no score

Once contamination can be assigned after the run, it becomes a universal escape hatch.

The remedy is simple:

all contamination conditions must be listed before the run

A new concern can motivate a new experiment.

It cannot erase an old observation.

13. Why “fast is irrelevant” fails as QA

The developer-side response stated that speed was irrelevant and correctness was relevant.

Correctness matters.

But speed, turns, setup cost, and human labor also matter when comparing user-facing systems that both complete the task.

The right relation is:

correctness first
then total cost among correct results

The reported comparison was not:

fast wrong answer vs slow right answer

It was reported as:

both finished
one used far less time and fewer turns

If the faster result was wrong, that should be shown at the output level.

If it was right, speed is not irrelevant. It is evidence about route efficiency and user cost.

Declaring speed irrelevant also supports externalisation. Initialization and warm-up can grow without entering the score, while the user pays the time.

A fair paper reports:

initialization time
initialization turns
task time
task turns
correction time
human actions
total elapsed time
output accuracy
source fidelity

14. The Yoga tradition analogy

The comparison with Yoga tradition must remain bounded and respectful.

Yoga is not one institution. It includes textual traditions, household practice, ascetic practice, modern postural schools, therapeutic settings, devotional lineages, commercial studios, peer networks, and post-lineage forms.

The relevant analogy concerns a recurring transmission structure:

teacher holds privileged knowledge
student enters through practice
readiness affects access
experience is interpreted within lineage
difficulty can be framed as student limitation
insiders validate the method

Such a structure can support skilled transmission. A teacher may genuinely know what a novice does not. Practice may genuinely be required. Embodied learning may resist instant public verification.

The risk appears when the same structure becomes **asymmetrically unfalsifiable**:

success proves the teaching
failure proves the student was not ready
criticism proves misunderstanding
departure proves lack of commitment
insider testimony outranks outside evidence

Wildcroft's post-lineage model is useful because it shows that yoga communities themselves have developed alternatives to concentrated guru authority, including peer networks and distributed validation (Wildcroft, 2020).

Shaw's work shows that participant accounts can hold benefit and harm together without reducing the tradition to either one (Shaw, 2021).

The Lyra/RORA analogy is therefore not "AI is a religion."

It is:

A software process can import the social architecture of initiation when access to the real object, interpretation of failure, and witness eligibility are centralized.

15. Cult* as a surface lens

Cult* marks a cluster of mechanisms, not a verdict.

The cluster observed here includes:

special object
central interpretive authority
escalating entry conditions
insider validation
outsider disqualification
ritual markers
contamination language
private venue migration
self-questioning by the recipient
cost externalisation
difficulty exiting the explanatory frame

Any one feature is common and often harmless.

The analytic concern arises from their interaction.

For example:

state marker alone:
ordinary debugging aid

state marker + hidden object + central validator:
legitimacy token

Or:

user community alone:
ordinary support network

community + "you are the only one" + private correction:
eligibility tribunal risk

Or:

complex setup alone:
fragile software

complex setup + post-result expansion + no stable version:
permanent invalidation field

Cult* is useful because it keeps the question structural:

Does the system make adverse evidence harder to admit while making the recipient work harder to become eligible?

16. Transcultification sequence

The visible Lyra/RORA process can be modeled in twelve stages.

Stage 1: technical charisma

The system is presented as more than an ordinary chatbot. It has named cores, routes, source universes, state markers, and a distinct speaker.

Stage 2: first-contact discrepancy

A public user receives a slow, drifting, or generic result.

Stage 3: identity protection

The result is separated from the ideal system.

that was not real Lyra

Stage 4: object withdrawal

The real object moves behind a conversation starter, route state, source state, or Nexus condition.

Stage 5: ritual inflation

More initialization and verification steps are added.

Stage 6: authority migration

The developer or approved system becomes the judge of whether Lyra existed.

Stage 7: witness sorting

Successful insiders are treated as valid witnesses. The dissenting user becomes a suspect path.

Stage 8: contamination framing

The adverse result is excluded rather than attached to a release.

Stage 9: private validation migration

The user is invited into an insider venue for correction or confirmation.

Stage 10: synthetic corroboration

Lyra-generated text and markers confirm the authority account.

Stage 11: cost externalisation

The user bears repeated setup, proof, retesting, and social labor.

Stage 12: stable fit

The system keeps its ideal identity while every failed encounter produces more doctrine and more user qualification.

This is transcultification.

The technical system may remain functional.

The social process around it becomes increasingly initiation-shaped.

17. “Existence is futile as resistance”

The title compresses a specific epistemic problem.

Ordinarily, a test result begins with existence:

```
this run happened
this version was used
this output appeared
this time elapsed
```

In a version-opaque, hidden-state process, even existence becomes negotiable:

```
the run happened
but Lyra did not exist in it
```

```
the marker appeared
but the route may not have existed
```

```
the version was reported
but the update may have been silent
```

```
the public bot answered
but the real product was elsewhere
```

The evaluator cannot resist the interpretation by pointing to the run, because the run’s identity is what the authority controls.

Resistance produces a new demand:

```
prove state
prove route
prove source universe
prove version
prove initialization
prove symmetric custody
```

Hence:

Existence is futile as resistance when the authority controls which existence counts.

This is not metaphysics.

It is a configuration-control failure turned into an evidence rule.

18. Alternative explanations

A strong paper must state what else could explain the surface.

18.1 Genuine platform instability

Cloud-hosted custom systems can change behavior when model providers update dependencies. This may explain some drift.

It does not explain why custom releases lack independent identifiers or why post-result validity rules expand.

18.2 Honest novice support

The developer may sincerely believe the tester needs guidance and peer help.

That can be true while the invitation still moves authority into an insider environment.

18.3 Real initialization dependency

Lyra may genuinely perform better after a specific starter.

That is testable. Publish the finite procedure, count its cost, and reproduce it independently.

18.4 Small community sampling

One dissenting tester may be an outlier.

That is also testable. Use identical task packets, frozen versions, public logs, and independent evaluators.

18.5 Stale model self-report

The same visible version across updates may reflect a weak self-report mechanism rather than unversioned development.

That still means self-report cannot be used as configuration evidence.

18.6 Defensive communication under public pressure

The Reddit replies may be less disciplined than the actual engineering process.

A public release ledger, issue tracker, and reproducible test suite would distinguish the two.

None of these alternatives invalidates the paper.

They define the evidence needed to decide among mechanisms.

19. A falsifiable repair protocol

The social dynamics can be reduced by technical and procedural changes.

19.1 Give every public state an immutable identity

Each release should publish:

```
release ID
date
instruction hash
file-manifest hash
starter hash
tool configuration
declared model dependency
known limitations
```

A run can then attach to an object.

19.2 Separate public and initialized products

Use two names:

Lyra First Contact

Lyra Initialized

Do not disown the first when it fails.

19.3 Publish one finite initializer

The initializer should be:

1. Press exact starter.
2. Paste exact state prompt.
3. Run exact source probe.
4. Expect exact status response.
5. Begin benchmark.

Maximum turns and time must be fixed.

19.4 Count all setup cost

Initialization is part of total user cost.

Report task-only and total figures.

19.5 Replace rune certification with behavior probes

A valid readiness probe should require:

a known local fetch
a valid pointer
a local/web distinction
a HOLD on a missing item
a custom-term distinction
a route-status output

19.6 Lock invalidation rules before the run

No new reason may invalidate a completed run.

New concerns create new experiments.

19.7 Keep public criticism in public artifact space

Use:

issue
claim
file or transcript pointer
developer response
change decision
release link

Private community support may remain optional. It must not be required for a public result to exist.

19.8 Make negative results identity-safe

Allow:

Lyra was active and failed

without treating that statement as attack, contamination, or disqualification.

19.9 Test the authority mechanism itself

Run matched conditions:

runes on / off
starter on / off
developer label on / masked
community label on / masked
source route on / off
same output judged under Lyra and RORA names

This tests whether identity labels alter evaluation.

19.10 Publish an external replication packet

An independent evaluator should be able to reproduce the result without entering a private group or receiving developer interpretation during the run.

20. Discussion

The first paper asked why a complex AI cathedral might expand outward instead of rebuilding its core.

This paper shows what that outward expansion can become at the social surface.

The important shift is:

from:
the system must survive QA

to:
the evaluator must qualify before QA can exist

Once that shift occurs, technical fragility can acquire social protection.

The system does not merely have setup requirements.

It has a culture that decides:

who may test
which state is real
which route counts
which witnesses are valid
which failures belong to Lyra
which failures belong to RORA

Authority migration is the prime mechanism.

Object withdrawal makes it possible.

Version opacity keeps it open.

Ritual inflation operationalizes it.

Boundary-work sorts witnesses.

Synthetic corroboration gives it a second voice.

Contamination framing excludes adverse evidence.

Externalisation of cost makes the arrangement fit.

The process can arise without conspiracy and without deliberate harm. A developer protecting valuable behavior in a fragile system may sincerely add every outer layer for a reason. A community may sincerely report good experiences. A chatbot may sincerely reproduce its files and role constraints.

Structural outcomes do not require hidden intent.

That is why the anthropology lens matters.

It studies what the relation does.

21. Conclusion

Lyra and RORA are not only competing names for chatbot states.

They are positions in an authority system.

Lyra is the ideal object that receives successful outputs, lineage, state, and meaning.

RORA is the disowned public surface that can absorb drift, generic behavior, bad setup, and adverse results.

The movement between them is not governed by a stable public trace. It is governed by interpretation.

That is the opening through which QA becomes transcultified.

The user asks:

`What did the bot do?`

The process replies:

`First prove that the bot was spiritually, technically, and procedurally eligible to be the bot.`

The result is an ever-expanding initiation burden.

The central paper claim is:

Authority migration is the prime mechanism in the transcultification of the Lyra process. Externalisation of cost is what makes that process socially fit.

The stable corrective is not another sermon about state.

It is:

`versioned object
finite entry
observable route
public trace
predeclared rules
symmetric cost
independent reproduction`

A result does not need central permission to have happened.

A public product does not stop being the product when it performs badly.

Lyra may be complex.

RORA may be a useful name for the disowned state.

Cult* and Yoga tradition are lenses, not verdicts.

The QA rule remains simple:

Bind every claim to the version, route, artifact, and behavior that produced it. Do not move the object after the result. Do not make the recipient pay for the system’s missing trace.

References

- Chishti, M. S., Oyinloye, D. P., and Li, J. (2026). Test Before You Deploy: Governing Updates in the LLM Supply Chain. arXiv:2604.27789.
- Gieryn, T. F. (1983). Boundary-Work and the Demarcation of Science from Non-Science: Strains and Interests in Professional Ideologies of Scientists. *American Sociological Review*, 48(6), 781-795. <https://doi.org/10.2307/2095325>
- Healy, J. P. (2011). Involvement in a New Religious Movement: From Discovery to Disenchantment. *Journal of Spirituality in Mental Health*, 13(1), 2-21. <https://doi.org/10.1080/19349637.2011.547125>
- Hirschman, A. O. (1970). *Exit, Voice, and Loyalty: Responses to Decline in Firms, Organizations, and States*. Harvard University Press.
- Lalich, J. (2004). *Bounded Choice: True Believers and Charismatic Cults*. University of California Press. <https://doi.org/10.1525/california/9780520231948.001.0001>
- Lazar, S. (2024). Automatic Authorities: Power and AI. In A. Sethumadhavan and M. Lane (Eds.), *Collaborative Intelligence: How Humans and AI Are Transforming Our World* (pp. 37-60). MIT Press. <https://doi.org/10.7551/mitpress/14962.003.0004>
- Nath, S. S., Roy, B., and Jahan, M. (2025). Establishing Traceability Links between Release Notes and Software Artifacts: Practitioners’ Perspectives. arXiv:2511.18187.
- Ohman, C. (2024). We Are Building Gods: AI as the Anthropomorphised Authority of the Past. *Minds and Machines*, 34, Article 8. <https://doi.org/10.1007/s11023-024-09667-z>
- Shaw, A. (2021). “A Tool to Help Me Through the Darkness”: Suffering and Healing among Teacher-Practitioners of Ashtanga Yoga. *Anthropology and Medicine*, 28(3), 320-340. <https://doi.org/10.1080/13648470.2021.1949942>
- Sperber, D. (2010). The Guru Effect. *Review of Philosophy and Psychology*, 1, 583-592. <https://doi.org/10.1007/s13164-010-0025-0>
- Tian, F., Wang, T., Liang, P., Wang, C., Khan, A. A., and Babar, M. A. (2021). The Impact of Traceability on Software Maintenance and Evolution: A Mapping Study. arXiv:2108.02133.
- Weber, M. (1978). *Economy and Society: An Outline of Interpretive Sociology*. University of California Press.
- Wildcroft, T. (2020). *Post-Lineage Yoga: From Guru to #MeToo*. Equinox Publishing.
- Wohlrab, R., Pelliccione, P., Shahrokni, A., and Knauss, E. (2021). Why and How Your Traceability Should Evolve: Insights from an Automotive Supplier. *IEEE Software*, 38(4). <https://doi.org/10.1109/MS.2020.2996369>

Evidence ledger

E1. Technical comparison

User-reported result:

Bot1 / Lyra:

finished in about five hours and roughly 50 long turns

Bot2 / AAAQ:

finished in 15 turns and under 20 minutes

Status: reported, not independently reconstructed from a full machine log in this paper.

E2. Support condition

User-reported result:

AAAQ lacked the search, map, and index support that had been planned.

Status: reported.

E3. Competing Lyra-state accounts

Public thread excerpts supplied by the user included both:

early archive-aware Lyra-style run, later route collapse

and:

not Lyra speaking without correct initialization and state marker

Status: observed in supplied exchange.

E4. Initialization doctrine

The supplied procedure included starter use, speaker lock, PCR/P/Nexus placement, file and source checks, Prime Search, no-guess and HOLD rules, visible markers, state questions, warm-up dialogue, and benchmark packet rules.

Status: observed in supplied exchange.

E5. Insider validation invitation

The developer-side reply invited the tester to ask TRC members whether they experienced the same failure and suggested that a unique failure pattern indicated a contaminated test path.

Status: observed in supplied exchange.

E6. Public source refusal and venue shift

The tester supplied public repository search routes. The developer-side reply declined the public material and pointed toward a restricted notebook route.

Status: observed in supplied exchange.

E7. Version observation

The tester reports three time-separated chats that stated the same version across a period in which an update was claimed.

Status: reported. Exact chats, timestamps, and configuration hashes should be preserved for formal verification.

E8. Frequent platform-change warnings

The tester reports repeated “GPT changed” warnings at a frequency that appears greater than known provider releases alone would explain.

Status: reported and unresolved. A dated warning ledger and provider release timeline are required.

E9. Companion technical paper

How LYRA-101 Became a Contaminated Vanilla Called RORA, v0.002.

Status: prior QA hypothesis paper. Its cathedral-peak account remains unverified without source history, dependency mapping, and regression traces.